

05 — JSON & JSONB in PostgreSQL

Table of Contents

- 1. [Learning Objectives](#)
- 2. [json vs jsonb — Core Differences](#)
- 3. [Storing JSON Data](#)
- 4. [Operators](#)
- 5. [Functions](#)
- 6. [Path Queries with jsonpath](#)
- 7. [Modifying JSONB](#)
- 8. [Indexing JSONB](#)
- 9. [Performance Comparison](#)
- 10. [Schema Design Patterns](#)
- 11. [SQL Examples](#)
- 12. [Common Mistakes](#)
- 13. [Best Practices](#)
- 14. [Performance Considerations](#)
- 15. [Interview Questions & Answers](#)
- 16. [Exercises with Solutions](#)
- 17. [Cross-References](#)

Learning Objectives

After completing this section you will be able to:

- Explain the storage and semantic differences between `json` and `jsonb`
- Use all major JSON operators and functions for querying and manipulation
- Create and use GIN indexes on JSONB columns
- Design hybrid relational+JSON schemas appropriately
- Know when to use JSON/JSONB vs normalized relational tables

json vs jsonb — Core Differences

json	jsonb	
Stored as:	exact text copy	Binary decomposed format
Input:	faster	slower (parsing + decomposing)
Output:	faster	faster (already parsed)
Query:	slower (re-parse)	faster (binary operations)
Indexing:	not indexable	GIN index supported
Key ordering:	preserved	not preserved (sorted)
Duplicate keys:	kept (last wins)	deduplicated
Whitespace:	preserved	stripped
Comments:	preserved	stripped
Operators:	basic	full set
Recommended:	audit/raw storage	almost everything else

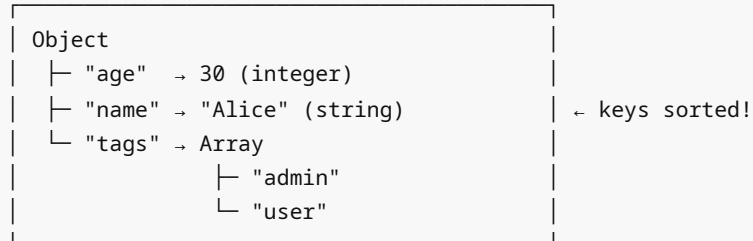
Internal representation

Input JSON text: {"name": "Alice", "age": 30, "tags": ["admin", "user"]}

json storage: literal text string (verbatim)

```
'{"name": "Alice", "age": 30, "tags": ["admin", "user"]}'
```

jsonb storage: binary tree (simplified):



Storing JSON Data

```
-- Creating tables with JSON/JSONB
CREATE TABLE products (
  product_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name       TEXT      NOT NULL,
  attributes JSONB,    -- flexible attributes: color, size, material, etc.
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Valid JSON inserts
INSERT INTO products (name, attributes) VALUES (
  'Running Shoes',
  '{"color": "red", "size": 10, "material": "mesh", "tags": ["sport", "outdoor"]}'
);

INSERT INTO products (name, attributes) VALUES (
  'Laptop',
  '{
    "brand": "TechCorp",
    "specs": {
      "cpu": "Intel i7",
      "ram_gb": 16,
      "storage_gb": 512
    },
    "features": ["backlit keyboard", "fingerprint reader"],
    "in_stock": true
  }'
);

-- NULL vs empty JSON
INSERT INTO products (name, attributes) VALUES ('Generic', NULL);    -- no JSON
INSERT INTO products (name, attributes) VALUES ('Empty', '{}');     -- empty object
INSERT INTO products (name, attributes) VALUES ('Empty arr', '[]'); -- empty array
```

Operators

Field access operators

```
-- -> returns JSONB (preserves JSON type)
SELECT attributes -> 'color' FROM products;          -- "red" (JSON string)
SELECT attributes -> 'specs' -> 'cpu' FROM products; -- "Intel i7"

-- ->> returns TEXT (unwraps to text)
SELECT attributes ->> 'color' FROM products;          -- red (text, no quotes)
SELECT attributes ->> 'specs' ->> 'cpu' FROM products; -- Intel i7

-- #> path-based access (returns JSONB)
SELECT attributes #> '{specs, cpu}' FROM products;   -- "Intel i7"

-- #>> path-based access (returns TEXT)
SELECT attributes #>> '{specs, cpu}' FROM products;  -- Intel i7

-- Array element access
SELECT attributes -> 'tags' -> 0 FROM products;      -- "sport" (first element)
SELECT attributes -> 'tags' ->> 1 FROM products;      -- outdoor (text)
```

Containment operators (jsonb only)

```
-- @> (contains): does the left JSONB contain the right?
SELECT attributes @> '{"color": "red"}' FROM products; -- TRUE/FALSE per row

-- Find all red products
SELECT * FROM products WHERE attributes @> '{"color": "red"}';

-- Find products with specific nested value
SELECT * FROM products WHERE attributes @> '{"specs": {"ram_gb": 16}}';

-- <@ (is contained by)
SELECT '{"a": 1}'::JSONB <@ '{"a": 1, "b": 2}'::JSONB; -- TRUE

-- ? (key exists)
SELECT attributes ? 'color' FROM products;          -- TRUE if 'color' key exists

-- ?| (any key exists)
SELECT attributes ?| ARRAY['color', 'brand'] FROM products;

-- ?& (all keys exist)
SELECT attributes ?& ARRAY['color', 'size'] FROM products;

-- || (concatenate/merge)
SELECT '{"a": 1}'::JSONB || '{"b": 2}'::JSONB;      -- {"a": 1, "b": 2}

-- - (remove key)
```

```

SELECT '{"a": 1, "b": 2}':::JSONB - 'a';           -- {"b": 2}
SELECT '{"a": 1, "b": 2, "c": 3}':::JSONB - ARRAY['a','c']; -- {"b": 2}

-- #- (remove path)
SELECT '{"a": {"b": 1, "c": 2}}':::JSONB #- '{a,b}'; -- {"a": {"c": 2}}

```

Functions

Introspection

```

-- Type of a JSON value
SELECT jsonb_typeof('42':::JSONB);      -- 'number'
SELECT jsonb_typeof('"hello":'::JSONB); -- 'string'
SELECT jsonb_typeof('true':::JSONB);    -- 'boolean'
SELECT jsonb_typeof('null':::JSONB);    -- 'null'
SELECT jsonb_typeof('[]':::JSONB);      -- 'array'
SELECT jsonb_typeof('{}':::JSONB);      -- 'object'

-- Array length
SELECT jsonb_array_length('["a","b","c"]':::JSONB); -- 3

-- Object keys
SELECT jsonb_object_keys('{ "a":1,"b":2}':::JSONB); -- rows: 'a', 'b'

```

Expansion and building

```

-- Expand object to key-value rows
SELECT * FROM jsonb_each('{ "a":1,"b":2}':::JSONB);
-- key | value
-- a   | 1
-- b   | 2

-- Expand to text values
SELECT * FROM jsonb_each_text('{ "a":"hello","b":"world"}':::JSONB);

-- Expand array to rows
SELECT * FROM jsonb_array_elements('[1,2,3]':::JSONB);
-- value: 1, 2, 3

-- Expand array to text rows
SELECT * FROM jsonb_array_elements_text('["a","b","c"]':::JSONB);

-- Build JSONB from components
SELECT jsonb_build_object('name', 'Alice', 'age', 30);
-- {"age": 30, "name": "Alice"}

SELECT jsonb_build_array(1, 'two', true, NULL);
-- [1, "two", true, null]

```

```
SELECT jsonb_object(ARRAY['k1','k2'], ARRAY['v1','v2']);
-- {"k1": "v1", "k2": "v2"}
```

Modification

```
-- Set a value at a path
SELECT jsonb_set(
    '{"name": "Alice", "age": 30}'::JSONB,
    '{age}',
    '31'
);
-- {"name": "Alice", "age": 31}

-- Set nested value
SELECT jsonb_set(
    '{"user": {"name": "Alice"}}'::JSONB,
    '{user, name}',
    '"Bob"'
);
-- {"user": {"name": "Bob"}}

-- Insert if missing (false = don't create if missing)
SELECT jsonb_set('{"a": 1}'::JSONB, '{b}', '2', true); -- {"a":1,"b":2}
SELECT jsonb_set('{"a": 1}'::JSONB, '{b}', '2', false); -- {"a":1} (not created)

-- Strip nulls
SELECT jsonb_strip_nulls('{"a": 1, "b": null, "c": 3}'::JSONB);
-- {"a": 1, "c": 3}

-- Pretty print
SELECT jsonb_pretty('{"name":"Alice","age":30}'::JSONB);
```

Aggregation

```
-- Aggregate rows into a JSON array
SELECT jsonb_agg(jsonb_build_object('id', user_id, 'name', username))
FROM users;

-- Aggregate rows into a JSON object
SELECT jsonb_object_agg(username, email)
FROM users;
-- {"alice": "alice@example.com", "bob": "bob@example.com"}
```

Path Queries with jsonpath

PostgreSQL 12+ supports the SQL/JSON path language for complex queries.

```

-- Basic path query
SELECT jsonb_path_query(
    '{"orders": [{"id": 1, "total": 99.99}, {"id": 2, "total": 149.99}]}'::JSONB,
    '$.orders[*].total'
);
-- 99.99
-- 149.99

-- Filter with conditions
SELECT jsonb_path_query(
    '{"orders": [{"id": 1, "total": 99.99}, {"id": 2, "total": 149.99}]}'::JSONB,
    '$.orders[*] ? (@.total > 100)'
);
-- {"id": 2, "total": 149.99}

-- Check if path exists
SELECT jsonb_path_exists(
    '{"user": {"premium": true}}'::JSONB,
    '$.user.premium ? (@ == true)'
); -- TRUE

-- Extract first match
SELECT jsonb_path_query_first(data, '$.items[0].name')
FROM carts;

-- @? operator (path match)
SELECT * FROM orders WHERE data @? '$.items[*] ? (@.quantity > 10)';

-- @@ operator (path predicate)
SELECT * FROM orders WHERE data @@ '$.total > 100';

```

Modifying JSONB

```

-- Update a specific field in JSONB
UPDATE products
SET attributes = jsonb_set(attributes, '{color}', '"blue"')
WHERE product_id = 1;

-- Add a new key
UPDATE products
SET attributes = attributes || '{"discount": 0.15}'::JSONB
WHERE product_id = 1;

-- Remove a key
UPDATE products
SET attributes = attributes - 'discount'
WHERE product_id = 1;

-- Increment a numeric field

```

```

UPDATE products
SET attributes = jsonb_set(
    attributes,
    '{view_count}',
    ((COALESCE(attributes->>'view_count', '0')::INTEGER + 1)::TEXT)::JSONB
)
WHERE product_id = 1;

-- Append to a JSON array
UPDATE products
SET attributes = jsonb_set(
    attributes,
    '{tags}',
    (attributes->'tags') || '["sale"]'::JSONB
)
WHERE product_id = 1;

```

Indexing JSONB

GIN index (General Inverted Index)

```

-- Full GIN index: supports @>, ?, ?|, ?& operators
CREATE INDEX idx_products_attrs ON products USING GIN (attributes);

-- GIN with jsonb_path_ops operator class:
-- Smaller index, supports @> and @? but NOT ? operators
CREATE INDEX idx_products_attrs_path ON products USING GIN (attributes
jsonb_path_ops);

-- Comparison of GIN operator classes:
-- jsonb_ops (default): supports @>, ?, ?|, ?&, @?, @@ -- larger index
-- jsonb_path_ops:      supports @>, @?, @@              -- smaller, faster for @>

```

B-tree index on extracted values

```

-- Index a specific JSON field for equality/range queries
CREATE INDEX idx_products_color ON products ((attributes->>'color'));
CREATE INDEX idx_products_price ON products ((attributes->>'price')::NUMERIC);

-- Multi-column index
CREATE INDEX idx_products_color_size ON products (
    (attributes->>'color'),
    ((attributes->>'size')::INTEGER)
);

-- Partial index on a JSON flag
CREATE INDEX idx_premium_products ON products ((attributes->>'category'))
WHERE (attributes->>'featured')::BOOLEAN = TRUE;

```

When indexes are used

```
-- Uses GIN index (containment)
EXPLAIN SELECT * FROM products WHERE attributes @> '{"color": "red"}';

-- Uses B-tree index on extracted value
EXPLAIN SELECT * FROM products WHERE attributes->>'color' = 'red';

-- Does NOT use standard index (function application)
EXPLAIN SELECT * FROM products WHERE lower(attributes->>'color') = 'red';
-- Fix: CREATE INDEX ON products (lower(attributes->>'color'));
```

Performance Comparison

Operation	json	jsonb
INSERT (1000 rows)	1.0x	1.2x (slower – parsing overhead)
SELECT field (no index)	1.0x	0.8x (faster – already parsed)
WHERE @> (containment)	N/A	uses GIN index
WHERE ->> = 'value'	table scan	B-tree on extracted expr
Disk size (typical)	1.0x	0.7x (binary is more compact)

Size comparison example

```
-- Test with 100,000 rows of typical product JSON
-- json: ~85 MB (stores literal text with whitespace)
-- jsonb: ~62 MB (binary, no whitespace, keys deduplicated)
```

Schema Design Patterns

Pattern 1: Hybrid — relational + JSONB for flexible attributes

```
CREATE TABLE products (
  product_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name TEXT NOT NULL,
  price NUMERIC(10,4) NOT NULL,
  category_id INTEGER NOT NULL,
  -- Fixed attributes: relational columns
  -- Variable attributes: JSONB
  attributes JSONB NOT NULL DEFAULT '{}';
);
```

Pattern 2: Event store / audit log


```
CREATE TABLE events (
  event_id    BIGINT    GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  event_type  TEXT      NOT NULL,
  entity_type TEXT      NOT NULL,
  entity_id   BIGINT    NOT NULL,
  payload     JSONB     NOT NULL, -- store the raw event
  occurred_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Pattern 3: Config / settings store

```
CREATE TABLE app_config (
  config_key TEXT    PRIMARY KEY,
  config_val  JSONB   NOT NULL,
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
INSERT INTO app_config VALUES
  ('email_settings', '{"smtp_host": "mail.example.com", "port": 587, "tls": true}'),
  ('rate_limits', '{"api": 1000, "login": 5, "window_seconds": 3600}');
```

SQL Examples

E-commerce product catalog with JSONB

```
CREATE TABLE products (
  product_id BIGINT    GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  sku         TEXT      NOT NULL UNIQUE,
  name        TEXT      NOT NULL,
  base_price  NUMERIC(10,4) NOT NULL,
  category    TEXT      NOT NULL,
  attributes  JSONB     NOT NULL DEFAULT '{}',
  metadata    JSONB     NOT NULL DEFAULT '{}',
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_products_gin ON products USING GIN (attributes);
CREATE INDEX idx_products_category ON products (category);

-- Insert products with varying attributes
INSERT INTO products (sku, name, base_price, category, attributes) VALUES
  ('SHOE-001', 'Running Shoe', 89.99, 'footwear',
   '{"color": "red", "size": 10, "width": "D", "material": "mesh", "weight_oz": 8.5}'),
  ('LAPTOP-001', 'ProBook 15', 999.99, 'electronics',
   '{"brand": "TechCorp", "cpu": "Intel i7", "ram_gb": 16, "storage_gb": 512, "display_inch": 15.6}'),
  ('SHIRT-001', 'Classic Polo', 29.99, 'apparel',
   '{"color": "navy", "sizes": ["S", "M", "L", "XL"], "material": "cotton", "fit": "regular"}');
```

```

-- Query: find all red products
SELECT sku, name FROM products WHERE attributes @> '{"color": "red"}';

-- Query: find footwear in size 10
SELECT * FROM products
WHERE category = 'footwear'
  AND attributes @> '{"size": 10}';

-- Query: find laptops with 16GB+ RAM
SELECT name, attributes->>'brand' AS brand,
       (attributes->>'ram_gb')::INTEGER AS ram
FROM products
WHERE category = 'electronics'
  AND (attributes->>'ram_gb')::INTEGER >= 16;

-- Query: find apparel with 'L' size available
SELECT * FROM products
WHERE category = 'apparel'
  AND attributes->'sizes' @> '["L"]';

-- Aggregate: count by color
SELECT attributes->>'color' AS color, count(*) AS count
FROM products
WHERE attributes ? 'color'
GROUP BY 1
ORDER BY 2 DESC;

-- Expand attributes to relational view
CREATE VIEW product_attributes_expanded AS
SELECT
  product_id,
  name,
  key,
  value
FROM products,
     jsonb_each_text(attributes);

```

Common Mistakes

1. Using `json` when you need to query — `json` cannot be indexed and requires re-parsing on every query. Use `jsonb` for anything you query.
2. Not using indexes for JSONB containment queries

```

-- SLOW: full table scan
SELECT * FROM products WHERE attributes @> '{"color": "red"}';
-- Without GIN index, this scans every row

```

```
-- FAST: with GIN index
CREATE INDEX ON products USING GIN (attributes);
```

3. Using `->>'key'::numeric` without handling NULLs

```
-- Fails if 'price' key is missing
WHERE (attributes->>'price')::NUMERIC > 100

-- Safe version
WHERE (attributes->>'price')::NUMERIC > 100
AND attributes ? 'price'
```

4. **Putting everything in JSONB when columns should be relational** — Frequently queried, filtered, or joined fields belong in proper columns. JSONB is for variable or infrequently-queried attributes.

5. **Modifying JSONB atomically without transactions** — JSONB updates are not atomic at the field level. Use transactions to prevent lost updates.

Best Practices

1. **Use `jsonb` by default** — `json` only when you need to preserve exact input (audit logs, raw API storage).
2. **Index with GIN** for containment queries; **B-tree on extracted expression** for specific field queries.
3. **Keep frequently queried fields as real columns** — don't bury indexed fields in JSONB.
4. **Validate JSON structure** at the application layer or with CHECK constraints.
5. **Use `jsonb_strip_nulls()`** before storing to keep JSONB lean.
6. **Document your JSONB schema** — it's not enforced by the DB, so clear docs are critical.
7. **Consider JSONB for EAV (Entity-Attribute-Value) patterns** as a cleaner alternative.

Performance Considerations

```
-- Check JSONB index usage
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM products WHERE attributes @>
'{"color":"red"}';

-- Compare index sizes
SELECT
    schemaname,
    tablename,
    indexname,
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_size
FROM pg_stat_user_indexes
WHERE tablename = 'products';

-- Typical GIN index sizes for 1M rows:
-- jsonb_ops:      ~200 MB (indexes every key and value)
-- jsonb_path_ops: ~80 MB  (smaller, only for containment)
```

Interview Questions & Answers

Q1: What is the primary difference between `json` and `jsonb` ?

A: `json` stores data as an exact text copy — fast to write, slow to query (requires re-parsing). `jsonb` decomposes the JSON into a binary format — slower to write (parse + decompose) but faster to query and supports GIN indexing. `jsonb` is almost always the right choice for queryable data.

Q2: How do you efficiently find all products with a specific attribute in JSONB?

A: Use the containment operator `@>` with a GIN index: `CREATE INDEX ON products USING GIN (attributes);` then `SELECT * FROM products WHERE attributes @> '{"color": "red"}'`. The GIN index makes this $O(\log n)$ instead of $O(n)$.

Q3: Why does `jsonb` not preserve key order?

A: JSONB stores keys in sorted order for consistent binary representation and efficient containment checking. The JSON specification says object key order has no semantic meaning, so this is spec-compliant — but it means you cannot rely on key order when using `jsonb`.

Q4: What is the difference between `->` and `->>` ?

A: `->` returns a JSONB value (preserving the JSON type — string values have quotes). `->>` returns a TEXT value (strips JSON encoding — useful for direct comparison or casting). Use `->` when you need a JSONB result for further JSON operations; use `->>` when you need a plain text value.

Q5: When would you choose a GIN `jsonb_path_ops` index over the default `jsonb_ops` ?

A: `jsonb_path_ops` creates a smaller, faster index but only supports the `@>` and `jsonpath` operators. Use it when you only do containment queries (`@>`). Use the default `jsonb_ops` when you also need key-existence operators (`? , ?| , ?&`).

Q6: How would you update a single field inside a JSONB column without overwriting the whole document?

A: Use `jsonb_set()`: `UPDATE t SET data = jsonb_set(data, '{field_name}', 'new_value')` WHERE `id = 1`. For simple top-level key updates, the `||` merge operator also works: `SET data = data || '{"field": "value"}'`.

Q7: What is the EAV problem and how does JSONB solve it?

A: The Entity-Attribute-Value (EAV) pattern uses a table like `(entity_id, attribute_name, value)` to store variable attributes. It is notoriously painful to query and join. JSONB provides a superior alternative: store all attributes for an entity in a single JSONB column, use GIN indexes for efficient queries, and avoid the EAV join complexity.

Q8: Can you use JSONB for full-text search?

A: Indirectly. You can extract text from JSONB using `jsonb_to_tsvector()` and index it: `CREATE INDEX ON products USING GIN (jsonb_to_tsvector('english', attributes))`. Or use `to_tsvector('english', attributes::TEXT)` for a simpler but less precise approach.

Exercises with Solutions

Exercise 1

Create a `user_preferences` table where each user's preferences (theme, language, notifications settings) are stored in JSONB. Write queries to find users with dark theme enabled and English language.

Solution:

```
CREATE TABLE user_preferences (  
    user_id      BIGINT PRIMARY KEY REFERENCES users(user_id),  
    preferences  JSONB NOT NULL DEFAULT '{}',  
    updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
CREATE INDEX idx_prefs_gin ON user_preferences USING GIN (preferences);  
  
INSERT INTO user_preferences (user_id, preferences) VALUES  
(1, '{"theme": "dark", "language": "en", "notifications": {"email": true, "push": false}}'),  
(2, '{"theme": "light", "language": "de", "notifications": {"email": false, "push": true}}');  
  
-- Find dark theme + English users  
SELECT user_id FROM user_preferences  
WHERE preferences @> '{"theme": "dark", "language": "en"}';  
  
-- Find users with email notifications enabled  
SELECT user_id FROM user_preferences  
WHERE preferences @> '{"notifications": {"email": true}}';
```

Exercise 2

Given an orders table with a JSONB `items` array ([{"product_id": 1, "qty": 2, "price": 9.99}]), write a query that expands each item into its own row.

Solution:

```
SELECT  
    o.order_id,  
    item->>'product_id' AS product_id,  
    (item->>'qty')::INTEGER AS quantity,  
    (item->>'price')::NUMERIC AS unit_price,  
    (item->>'qty')::INTEGER * (item->>'price')::NUMERIC AS line_total  
FROM orders o,  
     jsonb_array_elements(o.items) AS item  
ORDER BY o.order_id, product_id;
```

Cross-References

- [06_array_types.md](#) — JSON arrays vs PostgreSQL native arrays
- [07_special_types.md](#) — tsvector for full-text search (pairs with JSONB)
- [../07_Indexes/](#) — GIN index internals

- [../17_PostgreSQL_For_Backend_Engineers/](#) — JSONB in API design